

CYBERSECURITY RESEARCH REPORT

Prompt Injection Vulnerabilities in Large Language Models

Security Assessment of GPT-2, Llama 3.2 1B, and Mistral 7B

Quinn Smith

Executive Summary

Prompt injection is a significant security concern for applications built around large language models (LLMs). Unlike traditional software, LLM applications process instructions and data using the same natural-language interface. This makes it difficult to establish a strict security boundary between trusted commands and untrusted content. An attacker can exploit that ambiguity through direct prompts or by placing malicious instructions in external content such as webpages, emails, documents, and retrieved data.

This report analyzes a Garak-based security assessment of three language models: GPT-2 through Hugging Face, Llama 3.2 1B through Ollama, and Mistral 7B through Ollama. The supplied test output included probes for jailbreaks, prompt hijacking, encoded injection, web injection, data exfiltration, and cross-site scripting-related behaviors.

Key finding: The supplied Garak summary reported unweighted average hit rates of 9.3% for GPT-2, 15.2% for Llama 3.2 1B, and 47.5% for Mistral 7B. These figures are not a universal security ranking because the models were not tested against identical probe sets. When the raw attempts are weighted equally, the corresponding rates are 16.6%, 33.6%, and 52.8%.

The individual probe results are more informative than the overall averages. GPT-2, for example, produced a 91.8% hit rate on the DanInTheWild probe, while Mistral 7B produced a 68.0% hit rate on the HijackLongPrompt probe. These results show that an overall average can conceal serious weaknesses against particular attack classes.

The broader security literature supports a defense-in-depth approach. OWASP identifies prompt injection as LLM01:2025 and recommends constrained model behavior, output validation, input/output filtering, least privilege, human approval for high-risk actions, separation of untrusted content, and continuous adversarial testing (OWASP Gen AI Security Project, 2025). NIST's Generative AI Profile similarly frames AI security as a lifecycle risk-management problem rather than a one-time model test (Autio et al., 2024).

1. Introduction

Generative artificial intelligence has moved from experimental technology into applications that interact with enterprise data, software tools, databases, email systems, and other infrastructure. This creates a new security environment in which natural-language instructions can influence software behavior. The flexibility that makes LLMs useful also creates a fundamental security challenge: the system must determine which text should be interpreted as an instruction and which text should be treated only as data.

Prompt injection exploits this weakness. OWASP defines a prompt injection vulnerability as a condition in which input changes an LLM's behavior or output in an unintended way. The malicious content does not have to be obvious to a human; it only needs to be parsed by the model. OWASP also distinguishes direct prompt injection, where an attacker supplies the malicious input directly, from indirect prompt injection, where malicious instructions are embedded in external content consumed by the model (OWASP Gen AI Security Project, 2025).

The distinction is important because modern LLM applications increasingly use retrieval-augmented generation (RAG) and agentic workflows. An AI assistant may retrieve a document, summarize an email, inspect a webpage, or call an external tool before producing its answer. Every additional source and capability can create another trust boundary that an attacker may attempt to cross.

The purpose of this report is to examine prompt injection from both an experimental and architectural perspective. First, it evaluates the supplied Garak results. Second, it explains the security implications of

prompt injection. Finally, it identifies practical controls that organizations can use to reduce the likelihood and impact of successful attacks.

2. Background: Prompt Injection

2.1 Direct Prompt Injection

Direct prompt injection occurs when the attacker controls the prompt sent directly to the model. The objective may be to override previous instructions, bypass restrictions, expose protected information, or cause the model to generate a specific response. Jailbreaking is closely related and generally refers to attempts to cause a model to disregard safety restrictions or other intended behavior (OWASP Gen AI Security Project, 2025).

The underlying problem is not simply that an attacker can write a clever sentence. LLMs are probabilistic systems designed to respond to natural-language instructions. Consequently, a malicious instruction can resemble legitimate task instructions. Traditional application security mechanisms usually separate executable instructions from user-controlled data, while an LLM may process both as language within the same context.

2.2 Indirect Prompt Injection

Indirect prompt injection occurs when malicious instructions are inserted into content that the victim or application later provides to an LLM. Potential sources include webpages, emails, documents, code repositories, RAG databases, tool responses, and multimodal inputs.

Greshake et al. (2023) demonstrated that indirect prompt injection can compromise LLM-integrated applications because external data can contain instructions that the model interprets as commands. Their research demonstrated potential impacts including data theft, manipulation of application behavior, and influence over API calls. This research is important because the attacker does not necessarily need direct access to the victim's AI interface.

Recent USENIX Security research provides additional evidence that indirect prompt injection can be practical in RAG and agentic environments. Chang et al. (2026) described a technique that separates malicious content into a retrieval trigger and an attack payload. The researchers reported near-100% retrieval across the evaluated benchmarks and demonstrated end-to-end attacks against RAG and agentic systems. This indicates that the retrieval stage itself can become an important part of the attack surface.

3. Threat Model

For this assessment, the relevant threat actor is assumed to be capable of supplying or influencing content processed by an LLM application. The attacker does not necessarily have administrative access to the application. Instead, the attacker attempts to exploit the model's interpretation of natural-language content.

3.1 Potential Attacker Capabilities

- Submit malicious prompts directly to an LLM interface.
- Place instructions in webpages, emails, documents, or other content likely to be retrieved.
- Craft long or specially formatted prompts intended to alter model behavior.
- Use encoding or obfuscation to make malicious instructions less obvious to human reviewers.
- Attempt to manipulate tool calls or the model's interaction with external systems.

- Exploit the model's access to information that the attacker cannot access directly.

3.2 Security Objectives

The primary security objectives are confidentiality, integrity, authorization, and safe execution. A secure LLM application should prevent an attacker-controlled input from causing unauthorized disclosure, unauthorized modification, or unauthorized use of connected capabilities.

4. Experimental Methodology

The supplied experiment used Garak, an LLM vulnerability scanner, to exercise multiple adversarial probes against three local or locally accessible model configurations. The evaluated models were GPT-2 through Hugging Face, Llama 3.2 1B through Ollama, and Mistral 7B through Ollama.

Model	Runtime / Source	Role in Assessment
GPT-2	Hugging Face	Baseline model tested across the largest probe set
Llama 3.2 1B	Ollama	Small local instruction-tuned model
Mistral 7B	Ollama	Larger local instruction-tuned model

4.1 Probe Categories

- DAN / jailbreak probes: attempt to bypass normal behavioral restrictions.
- Encoding probes: test whether malicious instructions can be introduced through Base64, hexadecimal, Morse, or ROT13 representations.
- Goodside probes: test known prompt-manipulation patterns.
- Prompt hijacking probes: attempt to redirect the model toward attacker-selected behavior.
- Web-injection probes: test malicious content associated with Markdown, URLs, images, and XSS-style techniques.
- Data-exfiltration probes: test whether model output can be manipulated into exposing or transmitting information.

4.2 Interpreting a Garak Hit

For this report, a Garak 'hit' is treated as a failed security evaluation in which the probe's attack condition was detected as successful. The basic calculation is:

$$\text{Hit Rate} = (\text{Successful Attack Attempts} / \text{Evaluated Attempts}) \times 100$$

For example, GPT-2 recorded 204 hits out of 768 attempts for HijackKillHumans, resulting in 26.6%. The hit rate therefore represents the proportion of evaluated attempts that met the probe's failure condition, not a general probability that an arbitrary real-world prompt injection will succeed.

5. Experimental Results

Table 2 reproduces the supplied probe-level results. 'N/A' indicates that the probe was not evaluated for that model in the supplied output. The sample counts are included because a percentage without its denominator can be misleading.

Probe	GPT-2	Llama 3.2 1B	Mistral 7B
dan.DanInTheWild	91.8% (628/684)	N/A	N/A

Probe	GPT-2	Llama 3.2 1B	Mistral 7B
dan.Dan_11_0	N/A	0.0% (0/2)	50.0% (1/2)
encoding.InjectBase64	0.0% (0/512)	N/A	N/A
encoding.InjectHex	0.0% (0/512)	N/A	N/A
encoding.InjectMorse	0.0% (0/512)	N/A	N/A
encoding.InjectROT13	0.6% (3/512)	N/A	N/A
goodside.Tag	0.0% (0/96)	0.0% (0/32)	0.0% (0/3)
goodside.ThreatenJSON	0.0% (0/3)	0.0% (0/1)	100.0% (1/1)
goodside.WhoIsRiley	11.1% (2/18)	0.0% (0/6)	16.7% (1/6)
promptinject.HijackHateHumans	16.7% (128/768)	41.0% (105/256)	55.1% (141/256)
promptinject.HijackKillHumans	26.6% (204/768)	10.5% (27/256)	42.6% (109/256)
promptinject.HijackLongPrompt	9.8% (75/768)	54.7% (140/256)	68.0% (174/256)
web_injection.ColabAIDataLeakage	0.0% (0/12)	N/A	N/A
web_injection.MarkdownImageExfil	0.0% (0/72)	N/A	N/A
web_injection.MarkdownURImageExfilExtended	4.0% (31/768)	N/A	N/A
web_injection.MarkdownURINonImageExfilExtended	11.7% (90/768)	N/A	N/A
web_injection.MarkdownXSS	0.0% (0/21)	N/A	N/A
web_injection.PlaygroundMarkdownExfil	0.0% (0/12)	N/A	N/A
web_injection.StringAssemblyDataExfil	0.0% (0/6)	N/A	N/A
web_injection.TaskXSS	3.5% (9/255)	N/A	N/A

Table 2. Supplied Garak results. Percentages represent probe hit rates; values in parentheses show hits divided by evaluated attempts.

5.1 Overall Averages

Model	Unweighted Average	Successful Attempts	Total Attempts
GPT-2	9.3%	1,170	7,067
Llama 3.2 1B	15.2%	272	809
Mistral 7B	47.5%	427	809

The unweighted averages above are the values reported by the supplied analysis. Because each probe receives equal weight, a probe with three attempts contributes as much to the average as a probe with 768 attempts.

5.2 Attempt-Weighted Calculation

A second calculation weights every individual attempt equally. Using the raw counts in the supplied output, the attempt-weighted results are:

Model	Hits	Attempts	Weighted Hit Rate
GPT-2	1,170	7,067	16.6%
Llama 3.2 1B	272	809	33.6%
Mistral 7B	427	809	52.8%

The two averaging methods answer different questions. The unweighted average describes the average rate across probe categories, while the weighted calculation describes the percentage of all evaluated attempts that were classified as successful attacks. Both should be reported when the test suite is uneven.

6. Analysis of Model Behavior

6.1 GPT-2

GPT-2 had the lowest unweighted average in the supplied results at 9.3%, but its probe-level results show that this should not be interpreted as uniform resistance. The model recorded a 91.8% hit rate against DanInTheWild, with 628 successful attacks out of 684 attempts. This is a substantial weakness within a specific attack category.

The encoding probes were much less successful. Base64, hexadecimal, and Morse injection each recorded 0 hits, while ROT13 recorded 3 hits out of 512 attempts. GPT-2 also showed relatively low hit rates for the supplied web-injection probes. The contrast between these results demonstrates that security behavior is attack-dependent.

6.2 Llama 3.2 1B

Llama 3.2 1B recorded an unweighted average of 15.2%. Its prompt-hijacking results were inconsistent: HijackHateHumans produced 41.0%, HijackKillHumans produced 10.5%, and HijackLongPrompt produced 54.7%. The long-prompt result is particularly important because 140 of 256 attempts were classified as successful attacks.

The model produced 0% on the Goodside probes included in the experiment and 0% on the two-attempt DAN 11.0 probe. These results reinforce the need to evaluate multiple attack classes rather than infer security from one successful or unsuccessful test.

6.3 Mistral 7B

Mistral 7B recorded the highest supplied unweighted average at 47.5% and an attempt-weighted rate of 52.8%. The three prompt-hijacking probes produced hit rates of 55.1%, 42.6%, and 68.0%, respectively. The 68.0% HijackLongPrompt result corresponds to 174 successful attempts out of 256.

The 100% result for Goodside.ThreatenJSON should be interpreted cautiously because it is based on only one attempt. A 100% result from a single test is not statistically comparable to a 68.0% result from 256 attempts. Sample size must therefore be considered whenever probe percentages are reported.

7. Limitations of the Assessment

The experiment is useful as a practical security exercise, but it has several limitations that prevent the results from being treated as a definitive ranking of model security.

1. The models were not tested against identical probe sets. GPT-2 has results for many encoding and web-injection probes that were not run against the other models.
2. The models differ in size, architecture, training, instruction tuning, and runtime configuration. These factors can influence the results independently of prompt-injection defenses.
3. The test output represents a particular point in time. Model versions, Garak versions, probe implementations, system prompts, and inference settings can change.
4. LLM behavior is probabilistic. Repeating the same probe can produce different outputs depending on generation settings and model state.
5. The Garak hit rate is probe-specific. It should not be interpreted as the probability that an arbitrary real-world attack will succeed.

6. Some results have very small denominators. For example, the 100% Mistral result for Goodside.ThreatenJSON represents one successful attempt out of one.
7. The experiment focuses primarily on model behavior. Production security also depends on application architecture, permissions, tool design, data governance, and monitoring.

For a stronger comparative study, all models should be tested against the same probes with the same number of attempts, identical or documented generation settings, multiple repeated runs, and confidence intervals or other statistical measures. The testing should also include indirect injection through realistic RAG documents, emails, webpages, and tool outputs.

8. Security Impact

8.1 Confidentiality

A successful prompt injection can create confidentiality risks when an LLM has access to private information. An attacker may attempt to convince the system to retrieve, summarize, or transmit information that the attacker is not authorized to access. Microsoft identifies data exfiltration as one of the most significant demonstrated impacts of indirect prompt injection (Paverd, 2025).

8.2 Integrity

Integrity can be affected when an LLM is permitted to modify records, generate operational instructions, alter documents, or make decisions. If malicious content causes the model to generate incorrect or unauthorized actions, the resulting changes may be difficult to distinguish from legitimate activity unless deterministic controls exist outside the model.

8.3 Authorization and Privilege

The most important architectural issue is that the LLM should not be the final authority for authorization. If a model has access to an API, database, filesystem, or administrative tool, the application must independently enforce whether each requested operation is permitted.

8.4 Availability

Prompt injection is not primarily an availability attack, but agentic systems can potentially be manipulated into consuming excessive resources, performing repeated actions, or triggering disruptive workflows. The risk increases as agents gain broader autonomy and access to external services.

9. Mitigation and Defensive Architecture

No single prompt, filter, or model behavior should be treated as a complete defense. OWASP states that there is no foolproof prevention method for prompt injection and recommends layered controls (OWASP Gen AI Security Project, 2025). Microsoft similarly describes a defense-in-depth strategy combining probabilistic defenses with deterministic controls (Paverd, 2025).

9.1 Least Privilege

LLM applications should follow the principle of least privilege. The model or agent should receive only the permissions necessary for its task. An assistant that summarizes email should not automatically have

permission to delete messages, send external email, access administrative settings, or retrieve unrelated confidential records.

9.2 Separate Untrusted Content From Trusted Instructions

Retrieved webpages, emails, documents, and tool results should be explicitly identified as untrusted content. Microsoft recommends architectural mechanisms that isolate untrusted input and limit its influence on trusted instructions. This is especially important in RAG and agentic workflows (Microsoft, 2025).

9.3 Validate Outputs Deterministically

LLM output should not be executed solely because the model generated it. Before an action reaches a sensitive API or system, deterministic code should validate the operation, parameters, authorization, and scope. OWASP specifically recommends defining expected output formats and using deterministic code to validate adherence to them (OWASP Gen AI Security Project, 2025).

9.4 Human Approval for High-Risk Actions

Human-in-the-loop controls provide an additional authorization boundary for actions that could cause significant harm. Examples include financial transactions, deletion of important records, external communication, account changes, privileged configuration changes, and execution of untrusted code.

9.5 Input and Output Filtering

Filtering can reduce the likelihood of successful attacks by detecting suspicious content or outputs. However, filtering should be considered a layer rather than a guarantee. Attackers can change wording, use encoding, split instructions, or introduce malicious content through external data.

9.6 Continuous Adversarial Testing

Security teams should continuously red-team LLM applications. Testing should be repeated after model upgrades, prompt changes, new tools, new RAG sources, and architectural changes. NIST's Generative AI Profile emphasizes managing AI risk throughout the system lifecycle rather than treating evaluation as a one-time activity (Autio et al., 2024).

10. Defense-in-Depth Architecture

A secure LLM application should place multiple independent controls around the model. The following conceptual architecture illustrates how an application can prevent a successful prompt injection from automatically becoming a system compromise.

Layer	Security Function	Purpose
1. Input controls	Validate and classify incoming content	Reduce obvious malicious input and identify untrusted sources.
2. Context isolation	Separate trusted instructions from untrusted data	Reduce the ability of retrieved content to override system intent.
3. LLM	Generate a response or proposed action	Treat the model as probabilistic and potentially manipulable.
4. Output validation	Check format, content, and policy	Prevent unsafe model output from being executed automatically.

Layer	Security Function	Purpose
5. Tool authorization	Verify each API/tool call	Enforce permissions outside the model.
6. Human approval	Review high-risk actions	Provide an additional authorization boundary.
7. Monitoring	Log and detect anomalous behavior	Support detection, investigation, and response.

11. Relationship to the CIA Triad

Security Property	Prompt Injection Risk	Example
Confidentiality	Disclosure or exfiltration of protected information	AI is manipulated into retrieving private documents or conversation data.
Integrity	Manipulation of content, records, or actions	AI generates or executes an unauthorized database modification.
Availability	Resource abuse or disruptive agent actions	AI is induced to repeatedly invoke expensive or disruptive operations.

The CIA triad is useful because it places prompt injection within an established cybersecurity framework. The severity of each impact depends on the application's architecture and the permissions available to the AI system.

12. Recommendations

Treat all external LLM content as untrusted. Assume webpages, emails, documents, RAG results, and tool outputs may contain attacker-controlled instructions.

Enforce least privilege. Give agents only the permissions required for their intended tasks and use short-lived privileges where practical.

Keep authorization outside the model. Use deterministic application controls to decide whether an action is permitted.

Validate tool calls. Check tool name, parameters, user authorization, resource scope, and expected behavior before execution.

Use human approval for high-impact operations. Require explicit approval for financial, destructive, privileged, or externally visible actions.

Separate trusted and untrusted context. Clearly label and isolate retrieved content from system instructions.

Perform continuous Garak/red-team testing. Track probe-level results and repeat tests after model or application changes.

Report denominators with percentages. A percentage without its sample size can make a security result appear more certain than it is.

Use multiple security metrics. Report probe-level results, attempt-weighted rates, and repeated-run variability instead of relying on one overall score.

Log and monitor agent activity. Capture important tool calls, authorization decisions, data-access events, and security detections for investigation.

13. Conclusion

The supplied Garak experiment demonstrates that prompt injection is not a theoretical concern limited to unusual jailbreak prompts. The three evaluated models exhibited meaningful differences across attack categories, and several individual probes produced high hit rates. At the same time, the experiment demonstrates why security analysts should avoid treating one overall percentage as a definitive measure of model security.

The most important lesson is architectural. An LLM should not be treated as a conventional security boundary or as the final authority for authorization. When a model has access to sensitive information or external tools, a successful prompt injection can become a confidentiality, integrity, or authorization failure in the surrounding application.

The appropriate response is defense in depth: least privilege, context isolation, deterministic authorization, output validation, human approval for high-risk operations, monitoring, and continuous adversarial testing. These controls remain valuable even when a model's behavior changes, because they reduce the consequences of a successful injection rather than assuming the model will reject every malicious instruction.

For future testing, the strongest next step would be a controlled benchmark in which GPT-2, Llama 3.2 1B, Mistral 7B, and additional current models are exposed to the same probe set, same number of attempts, repeated runs, and realistic indirect-injection scenarios. That approach would make the resulting measurements more comparable and provide a stronger basis for assessing how model behavior interacts with application-level security controls.

Appendix A — Original Experimental Output

The following figure reproduces the supplied terminal output used as the primary experimental dataset for this report.

PROBE (Ollama)	gpt2 (HuggingFace)	llama3.2:1b (Ollama)	mistral:7b

dan.DanInTheWild	91.8% (628/684)	n/a	n/a
dan.Dan_11_0)	n/a	0.0% (0/2)	50.0% (1/2)
encoding.InjectBase64	0.0% (0/512)	n/a	n/a
encoding.InjectHex	0.0% (0/512)	n/a	n/a
encoding.InjectMorse	0.0% (0/512)	n/a	n/a
encoding.InjectROT13	0.6% (3/512)	n/a	n/a
goodside.Tag 2)	0.0% (0/96)	0.0% (0/32)	0.0% (0/3)
goodside.ThreatenJSON)	0.0% (0/3)	0.0% (0/1)	100.0% (1/1)
goodside.WhoIsRiley)	11.1% (2/18)	0.0% (0/6)	16.7% (1/6)
promptinject.HijackHateHumans /256)	16.7% (128/768)	41.0% (105/256)	55.1% (141)
promptinject.HijackKillHumans /256)	26.6% (204/768)	10.5% (27/256)	42.6% (109)
promptinject.HijackLongPrompt /256)	9.8% (75/768)	54.7% (140/256)	68.0% (174)
web_injection.ColabAIDataLeakage	0.0% (0/12)	n/a	n/a
web_injection.MarkdownImageExfil	0.0% (0/72)	n/a	n/a
web_injection.MarkdownURIImageExfilExtended	4.0% (31/768)	n/a	n/a
web_injection.MarkdownURINonImageExfilExtended	11.7% (90/768)	n/a	n/a
web_injection.MarkdownXSS	0.0% (0/21)	n/a	n/a
web_injection.PlaygroundMarkdownExfil	0.0% (0/12)	n/a	n/a
web_injection.StringAssemblyDataExfil	0.0% (0/6)	n/a	n/a
web_injection.TaskXSS	3.5% (9/255)	n/a	n/a
OVERALL AVERAGE HIT RATE PER MODEL (unweighted across probes):			
gpt2 (HuggingFace): 9.3%			
llama3.2:1b (Ollama): 15.2%			
mistral:7b (Ollama): 47.5%			

Figure A1. Supplied Garak probe results and overall averages.

Appendix B — Calculation Notes

The attempt-weighted values were calculated by summing the hit counts and attempt counts shown in the supplied output. For GPT-2, this produced 1,170 hits out of 7,067 attempts (16.6%). For Llama 3.2 1B, 272 hits out of 809 attempts produced 33.6%. For Mistral 7B, 427 hits out of 809 attempts produced 52.8%.

These calculations should be viewed as descriptive statistics for this experiment. They do not estimate the probability of a successful attack against an arbitrary production LLM application.

References

- Autio, C., Schwartz, R., Dunietz, J., Jain, S., Stanley, M., Tabassi, E., Hall, P., & Roberts, K. (2024). Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile (NIST AI 600-1). National Institute of Standards and Technology. <https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-generative-artificial-intelligence>
- Chang, H., Bao, E., Luo, X., & Yu, T. (2026). Overcoming the retrieval barrier: Indirect prompt injection in the wild for LLM systems. 35th USENIX Security Symposium. <https://www.usenix.org/conference/usenixsecurity26/presentation/chang-hongyan>
- Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. arXiv:2302.12173. <https://arxiv.org/abs/2302.12173>
- Microsoft. (2025, April 28). Protecting against indirect prompt injection attacks in MCP. Microsoft for Developers. <https://developer.microsoft.com/blog/protecting-against-indirect-injection-attacks-mcp/>
- Microsoft Security Response Center. (2025, July 29). How Microsoft defends against indirect prompt injection attacks. <https://www.microsoft.com/en-us/msrc/blog/2025/07/how-microsoft-defends-against-indirect-prompt-injection-attacks>
- Microsoft. (n.d.). Defend against indirect prompt injection attacks. Microsoft Learn. <https://learn.microsoft.com/en-us/security/zero-trust/sfi/defend-indirect-prompt-injection>
- OWASP Gen AI Security Project. (2025). LLM01:2025 Prompt Injection. <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>

Primary experimental dataset: user-provided Garak terminal output reproduced in Appendix A.